

IMPLEMENTING OOP CONCEPTS IN C#

Learning Objectives

- 3.1 INTRODUCTION
- 3.2 INTRODUCTION TO CLASSES
 - 3.2.1 Access Modifiers
 - 3.2.2 Fields
 - 3.2.3 Methods
- 3.3 INTRODUCTION TO OBJECTS
- 3.4 CONSTRUCTOR
- 3.5 DESTRUCTORS
- 3.6 INHERITANCE
- 3.7 EXCEPTION HANDLING
- REVIEW QUESTIONS

3.1 INTRODUCTION

In this chapter, we investigate object orientation in C#. In procedural programming languages (like C), programming tends to be action oriented. C# programming, however, is object oriented. In C, the unit of programming is the function (functions are called methods in C#). In C#, the unit of programming is the class. Objects eventually are instantiated (i.e., created) from these classes and functions are encapsulated within the "boundaries" of classes as methods.

By contrast, C# programmers concentrate on creating their own user-defined types, called classes. We also refer to classes as programmer-defined types. Each class contains both data and a set of methods that manipulate the data. The data components, or data members, of a class are called member variables, or instance variables (many C# programmers prefer the term fields). Just as we call an instance of a built-in type—such as `int`—a variable, we call an instance of a user-defined type (i.e., a class) an object.

In C#, attention is focused on classes, rather than on methods. The nouns in a system-requirements document help the C# programmer determine an initial set of classes with which

to begin the design process. Programmers use these classes to instantiate objects that work together to implement the system. This chapter explains how to create and use classes and objects, a subject known as object-based programming (OBP). We will discuss inheritance and polymorphism—key technologies that enable object-oriented programming (OOP). Although we do not discuss inheritance in detail, it is part of several class definitions in this chapter.

3.2 INTRODUCTION TO CLASSES

A class is a collection of related objects. For example, a Fruit class consists of objects like mangoes, apple, banana etc. It is also called as a user defined data type. A class defines the properties and methods of the objects. It consists of fields (data) and methods (procedure or function) that operate on data. The basic syntax for a class definition is as follows:

```
Syntax:  class ClassName
        {
            [fields declaration]    //Note: Statements in [ ] are optional
            [methods definitions]
        }
```

Example: A Room class with no fields and methods

```
class Room
{
    // My Room class
}
```

3.2.1 Access Modifiers

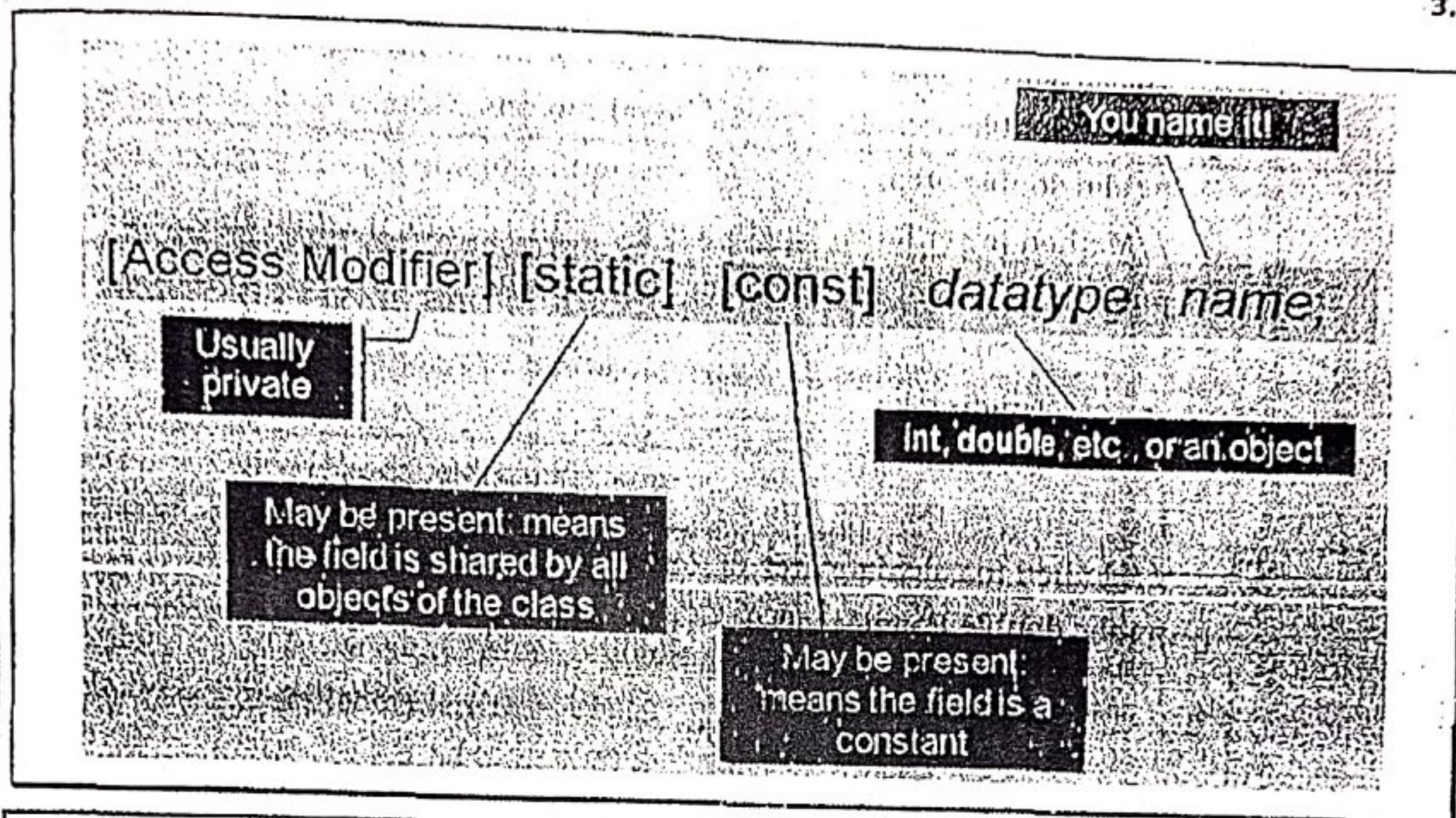
C# provides access modifiers to restrict access to the members of the class. Access modifiers are responsible for the visibility of the class members to an object and even to the inherited class. Access modifiers or accessibility levels can be applied to any member of the class. Various access modifiers provided by C# are described in Table 3-A.

Table 3-A

Access Modifier	Description
public	Makes a member accessible from anywhere.
protected	Makes a member accessible within the class and all classes derived from it only.
private	Makes a member accessible only within a class.
internal	Can be accessed only within the current project.

3.2.2 Fields

A field is used to encapsulate data. Fields are also called instance variables.



Example: Adding Fields to Room class

```
class Room
{
    public double length;    // Length of the Room
    public double breadth;   // Breadth of the Room
    public double area;      // Area of the Room
}
```

3.2.3 Methods

All C# programs constructed from a number of classes and those classes contain methods that solve the problems for the program. So a method is a kind of building blocks that solve a small problem.

A class with only data fields has no life. Objects created by such a class cannot respond to any messages. Methods are declared inside the body of the class but immediately after the declaration of data fields. The general form of a method declaration is:

Syntax: `[Access Modifier] ReturnType MethodName ([parameters-list])`

```
{
    Method-body;
}
```

Example: Adding Methods to Room class

```
class Room
```



```

public double length;        // Length of the Room
public double breadth;       // Breadth of the Room
public double area;          // Area of the Room
//Method to set Room length and breadth
public void RoomDimensions(double l, double b)
{
    length = l;
    breadth = b;
}
//Method to calculate area of Room
public double RoomArea()
{
    area = length * breadth;
    return area;
}
//Method to display Room Dimensions and Area
public void ShowRoomDetails()
{
    Console.WriteLine("Length of the Room: {0}", length);
    Console.WriteLine("Breadth of the Room: {0}", breadth);
    Console.WriteLine("Area of the Room: {0}", area);
}
}

```

3.3 INTRODUCTION TO OBJECTS

An object is a runtime entity. Objects are known as the instance of the class. Each object has its own properties. All the objects are uniquely identified. An object can be created as follows:

Syntax: `ClassName ObjectName=new ClassName([arguments]);`

Example: `Room objRoom1=new Room();`

3.3.1 Defining Objects

We can define objects as follows:

Example: `Room objRoom1;`

Here, object `objRoom1` points to nothing (Null Reference)

3.3.2 Creating Objects

Objects are created dynamically using the **new** keyword.

Example: `objRoom1=new Room();`

Now, object `objRoom1` refers to the Room class object.

3.2.3 Accessing Fields and Methods using Objects

Fields and methods of a class can be accessed using the objects of the class. To access any field or method of the class we use dot (.) operator with its objects.

Syntax: `ObjectName.FieldName = value;`
`ObjectName.MethodName([parameter-list]);`

Example: `//Accessing Fields - initialize Length and Breadth of Room`
`objRoom1.length = 15;`
`objRoom1.breadth = 20.5;`
`objRoom1.area= objRoom1.length * objRoom1.breadth;`
`//Accessing Method - display all details of Room`
`objRoom1.ShowRoomDetails();`

All the above mentioned concepts can be more understandable by grouping them into a program. So, let's take a look at following Program

Description of the Program: In this program, we are going to demonstrate how a class can be defined, how to fields and methods to it, how to create objects and how to access fields and methods of the class.

Step-1: Click on **File** menu and select **New >> Project**.

Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application**.

Step-3: Name the Application "**AddingClasses**" and set location.

Step-4: Add the following code to the C# Console Application.

`using System;`

`namespace AddingClasses`

`{`

`//Defining a class named Room`

`class Room`

`{`

`//Adding Fields to the Room class`

`public double length;      // Length of the Room`

`public double breadth;    // Breadth of the Room`

3.6

```

public double area;           // Area of the Room
//Adding Methods to the Room class
//Method to set Room length and breadth
public void RoomDimensions(double l, double b)
{
    length = l;
    breadth = b;
}
//Method to calculate area of Room
public double RoomArea()
{
    area = length * breadth;
    return area;
}
//Method to calculate area of Room
public void ShowRoomDetails()
{
    Console.WriteLine("Length of the Room: {0}", length);
    Console.WriteLine("Breadth of the Room: {0}", breadth);
    Console.WriteLine("Area of the Room: {0}", area);
}
} //End of Room class

```

/*-----Another Class containing Main()-----*/

```

class Program
{

```

```

    static void Main(string[] args)
    {

```

```

        /*---Case-1: Accessing Fields---*/

```

```

        //Creating object of the Room class: objRoom1

```

```

        Room objRoom1 = new Room();

```

```

        //Accessing fields: Set Room's length and breadth

```

```

        objRoom1.length = 15;

```

```

        objRoom1.breadth = 20.5;

```

```

        //Calculate area of the room

```

```

        objRoom1.area = objRoom1.length * objRoom1.breadth;

```

```

        //Display Details of Room-1

```



```

Console.WriteLine("-----[Room-1 : Accessing Fields]-----");
Console.WriteLine("Length of the Room: {0}", objRoom1.length);
Console.WriteLine("Breadth of the Room: {0}", objRoom1.breadth);
Console.WriteLine("Area of the Room: {0}", objRoom1.area);

/*---Case-2: Accessing Methods---*/
//Creating another object of the Room class: objRoom2
Room objRoom2 = new Room();
//Accessing methods
//Set Room's length and breadth using method RoomDimensions()
objRoom2.RoomDimensions(25, 30);
//Calculate area of the room using method RoomArea()
double areaRoom2 = objRoom2.RoomArea();
//Display Details of Room-2 using method ShowRoomDetails()
Console.WriteLine("\n\n-----[Room-2 : Accessing Methods]-----");
objRoom2.ShowRoomDetails();
}
}

```

Step-5: Press Ctrl + F5. The output window will appear as follows:

```

C:\WINDOWS\system32\cmd.exe
-----[Room-1 : Accessing Fields]-----
Length of the Room: 15
Breadth of the Room: 20.5
Area of the Room: 307.5

-----[Room-2 : Accessing Methods]-----
Length of the Room: 25
Breadth of the Room: 30
Area of the Room: 750
Press any key to continue

```

3.4 CONSTRUCTOR

A **constructor** is a special method that is used to initialize newly created objects. It can be used to initialize the objects to required, or default values at the time of object creation. It is **not mandatory** for the coder to write a constructor for the class. If no user defined constructor is provided for a class, compiler initializes objects of the class to the default value. Some of the are as follows:

- Numeric data types are set to 0.
- char data types are set to null character ("\\0").
- Reference variables are set to null.

3.4.1 Features of Constructors

The constructor methods have some special features. Such are as follows:

- Constructors should be public.
- Constructors need not to be invoked, they invoked automatically when ever the object of the class is created.
- Constructors do not have any return type not even void.
- Constructors have the same name as the class.
- Default parameters are not allowed.

3.4.2 Types of Constructors

C# provides following types of constructors:

- Implicit Default Constructor and Constructor with no arguments.
- Parameterized Constructor.
- Multiple Constructor/Constructor Overloading.
- Copy Constructor.

3.4.2.1 Implicit Default Constructor and Constructor with no arguments

You don't have to provide any constructors for your class. The compiler automatically provides a no-argument, default constructor for any class having no constructors. It initializes all instance variables of the class with some default values (0, '\\0', null, or false). But, If you write a constructor, however, with some parameters, and you don't write any default constructor, then C# does not provide an Implicit default constructor. A user defined default constructor is a constructor with no arguments. It is used to initialize values as per the user requirements.

Syntax: Constructor with no Arguments

```
class ClassName
{
    Fields Declaration;
    public ClassName()           //Constructor with no arguments
    {
        //body of the constructor
    }
}
```


Example: Constructor with no Arguments

```
class Rectangle
{
    double length, breadth, area;
    public Rectangle() //Constructor with no arguments
    {
        length=10;
        breadth=50;
        area=length*breadth;
    }
    public void Display()
    {
        Console.WriteLine("Length of Rectangle : "+length);
        Console.WriteLine("Breadth of Rectangle : "+breadth);
        Console.WriteLine("Area of Rectangle : "+area);
    }
}
```

Accessing: Constructor with no Arguments

```
class AccessConstructor
{
    public static void Main(String[] ar)
    {
        Rectangle rect1 = new Rectangle();
        rect1.Display();
    }
}
```

3.4.2.2 Parameterized Constructor

A Constructor that receive some information when it is invoked is called as parameterized constructor.

Syntax: Parameterized Constructor

```
class ClassName
```

```
{
```

```
    Fields Declaration;
```



```
public ClassName(ParameterList) //Parameterized Constructor
{
    //body of the constructor
}
```

Example: Parameterized Constructor

```
class Rectangle
{
    double length,breadth,area;
    //Parameterized Constructor
    public Rectangle(double l,double b)
    {
        length=l;
        breadth=b;
        area=length*breadth;
    }

    public void Display()
    {
        Console.WriteLine("Length of Rectangle : "+length);
        Console.WriteLine("Breadth of Rectangle : "+breadth);
        Console.WriteLine("Area of Rectangle : "+area);
    }
}
```

Accessing: Parameterized Constructor

```
class AccessConstructor
{
    public static void Main(String[] ar)
    {
        Rectangle rect1 = new Rectangle(20.5,30.5);
        rect1.Display();
    }
}
```


3.4.2.3 Multiple Constructor/ Constructor Overloading

A class can contain more than one constructor which means a class can contain multiple constructors. It is possible only if all the constructors of a class must have different number of arguments or with different types of arguments.

Syntax: Multiple Constructor

```
class ClassName
{
    Fields Declaration;
    public ClassName (Parameter list1)
    {
        //body of the constructor
    }
    public ClassName (Parameter list2)
    {
        //body of the constructor
    }
    .
    .
    .
    public ClassName (Parameter listN)
    {
        //body of the constructor
    }
}
```

Example: Multiple Constructor

```
class Volume
{
    double cuboid, sphere;
    public Volume(double l, double w, double h)
    {
        cuboid=l*w*h;
        Console.WriteLine("Volume of Cuboid = "+cuboid);
    }

    public Volume(double r)
```


3.12

```
{
    sphere=(4/3.0)*3.14*r*r*r;
    Console.WriteLine("Volume of Sphere = " + sphere);
}
```

Accessing: Multiple Constructor

```
class AccessConstructor
{
    public static void Main(String[] ar)
    {
        Volume cuboid = new Volume(10, 20, 30);
        Volume sphere = new Volume(2.0);
    }
}
```

3.4.2.4 Copy Constructor

Copying of objects is achieved by the use of a copy constructor. Like C++, C# also supports copy constructor. But, unlike C++, C# doesn't create an implicit default copy constructor if you don't write your own. A **copy constructor** is a special constructor for creating a new Object, as a copy of an existing object.

Syntax: Copy Constructor

```
class ClassName
{
    Fields Declaration;
    public ClassName(Parameter list1)
    {
        //body of the constructor
    }
    public ClassName (ClassName object) //Copy Constructor
    {
        //body of the constructor
    }
}
```

Example: Copy Constructor

```
class Copy
{
    ...
}
```



```
int a, b;
public Copy(int x, int y)
{
    a = x;
    b = y;
}
public Copy(Copy obj)    //Copy Constructor
{
    a = obj.a;
    b = obj.b;
}
public void Display()
{
    Console.WriteLine("A="+a);
    Console.WriteLine("B="+b);
}
}
```

Accessing: Copy Constructor

```
class AccessConstructor
{
    public static void Main(String[] ar)
    {
        Copy obj1 = new Copy(10, 20);
        Copy obj2 = new Copy(obj1);
        obj2.Display();
    }
}
```

3.5 DESTRUCTORS

The destructor is called when the object of the class is destroyed but this is not the case with the C#. The destructor of C#, instead, is called by the Garbage Collector and the garbage collector performs a clean up only at regular intervals or when the memory state requires it such as when the memory is full.

Syntax: Destructors

```
class ClassName
```



```

~ClassName ()    //Destructor

```

```

{
    //body of the constructor
}

```

Example: Destructors

```

class Copy

```

```

{
    public Copy()

```

```

    {
        Console.WriteLine("Constructor Invoked!");
    }

```

```

    ~Copy()    //Destructor

```

```

    {
        Console.WriteLine("Object Destroyed");
    }
}

```

Accessing: Destructors

```

class AccessDestructor

```

```

{
    public static void Main(String[] ar)

```

```

    {
        Copy obj1 = new Copy();

```

```

        {
            Copy obj2 = new Copy();

```

```

        }
        Copy obj3 = new Copy();
    }
}

```

3.6 INHERITANCE

Object Oriented Programming enables you to reuse an existing class to build a new class. This mechanism is known as Inheritance. Inheritance is the process by which the object of one class acquires the properties of the object of another class. We can take an existing class and

add additional features to it without modifying the original (Base) class. There are various advantages of using inheritance such as follows:

- Easy to write new programs by assembling existing pieces of code.
- We do not have to start from scratch.
- It speeds up development time.
- Only the new functionality code is to be written which means less bugs and cost.

C# supports **single** inheritance. C# does not support **multiple** inheritance. Multiple inheritance cause major confusion when applications get complex and are also considered to be inefficient programming in terms of memory overload. But we can implement **multiple interfaces** in C#. Let us see how we can implement single inheritance in C#.

A **single inheritance** contains one base class (supper class or parent class) and one derived class (sub class or child class). In this derived class may inherit some or all the properties of the base class.

Syntax: Single Inheritance in C#

```
class BaseClassName
{
    [fields declaration]
    [methods definitions]
}

class DerivedClassName : BaseClassName
{
    [fields declaration]
    [methods definitions]
}
```

Example: Demonstrating how to implement Single inheritance in C#

```
class Base
{
    public void Show()
    {
        Console.WriteLine("Base Class method invoked!");
    }
}

class Derived:Base //Inheriting class Base into class Derived
{
    public void Display()
    {
    }
}
```



```

        Console.WriteLine("Derived Class method invoked!");
    }
}
class SingleInheritance //class containing Main() method
{
    public static void Main(String[] ar)
    {
        Derived obj = new Derived();
        //invoking Base class method using Derived class object
        obj.Show();
        obj.Display(); //invoking derived class method
    }
}

```

3.6.1 Sealing a Class

If we do not want any other class to inherit from a particular class then we can **seal** that particular class. It means that a sealed class can not be inherited by any other class. We can seal any class by using **sealed** keyword.

Syntax: Sealing a class

```

sealed class ClassName
{
    [fields declaration]
    [methods definitions]
}

```

3.6.2 Creation of Base Class with the base Keyword

Whenever we instantiate the derived class, the compiler first instantiates the base class by calling its default constructor automatically (Implicitly). Then the compiler calls the constructor of the derived class. But, if there is a parameterized constructor in base class then that can not be invoked automatically by compiler. For this, we have to call it explicitly inside the derived class using the keyword **base**. The keyword **base** must be used with the constructor of the derived class. We can also call default constructor of the base class explicitly using the keyword **base**.

Note: Whenever we made an explicit call to the any of the base class constructor from derived class, the implicit calls will not work.

Syntax: Calling constructor of base class in derived class

```

class DerivedClassName : BaseClassName
{

```



```

public DerivedClassName([ParametersList]): base([ArgumentList])
{
    //body of the Derived Class constructor
}

```

Example: Calling constructor of base class in derived class

```

class Base
{
    public Base(int x,int y)
    {
        Console.WriteLine("Base Class Constructor Invoked!");
        Console.WriteLine("Base X= "+x+"Base Y="+y);
    }
}

class Derived:Base
{
    public Derived():base(5,10) //Explicit Call To Base Class Constructor
    {
        Console.WriteLine("Derived Class Constructor Invoked!");
    }
}

class InvokeConstructor
{
    public static void Main(String[] ar)
    {
        Derived obj = new Derived();
    }
}

```

3.7 EXCEPTION HANDLING

An exception is a condition that is caused by the runtime error in the program. A program may compile successfully but may not run properly. Such program may produce wrong results due to wrong logic or may terminate due to runtime errors such as, **dividing an integer by 0**.

Errors may arise anytime during the program execution for any reason, such as, the user

might give wrong input to the program. A robust program would check for these kinds of errors using error-handling techniques but checking for all these errors becomes unbearably complex using error-handling techniques. So, **Exceptions** comes into the picture. It simplifies control flow and makes programs robust.

To trap these exceptions or to handle these exceptions, C# provides three constructs, namely **try**, **catch** and **finally**. The **try** encloses the statements that might throw an exception whereas **catch** handles an exception if one exists. The **finally** can be used for doing any clean up process.

3.7.1 Exception Handling using try and catch Blocks

Try and Catch block help in handling raised exceptions. These blocks provide a structured way to handle runtime errors. Its syntax can be given as follows:

Syntax: Exception handling using try....catch Blocks

```
try
{
    //Program Logic
}
catch(ExceptionType ExceptionObject)
{
    //Error Handling Code
}
```

Example: Exception handling using try....catch Blocks

class HandlingExceptions

```
{
    public static void Main(String[] ar)
    {
        int x = 10, y = 0, res=0;
        try
        {
            res = x / y;        // Runtime Error: divide by 0
            Console.WriteLine("Result is : "+res);
        }
        catch (DivideByZeroException e)
        {
            Console.WriteLine("Divide by 0");
        }
        Console.WriteLine("End of the program.");
    }
}
```


As shown in the example the error-handling code is separated from the program logic, the program logic is written inside the **try block** and the error handling code is within the **catch block**. The catch block captures the exceptions thrown by the try block.

3.7.2 Multiple catch Blocks

Sometimes, it becomes necessary to have more than one catch block for a single try block as each catch block can have one class of exception. If we need to handle more than one exception in a try block, we need to have more than one catch block.

Note: Remember that more specialized catch block should come before a generalized one. Otherwise the compiler will show a compilation error.

Syntax: Multiple catch Blocks

```
try
{
    //Program Logic
}
catch(ExceptionType1 ExceptionObject)
{
    //Error Handling Code
}
catch(ExceptionType2 ExceptionObject)
{
    //Error Handling Code
}
.
.
.
catch(ExceptionTypeN ExceptionObject)
{
    //Error Handling Code
}
```

Example: Multiple catch Blocks

```
class HandlingExceptions
{
    public static void Main(String[] ar)
    {
        int x = 10, y = 0, res=0;
```


3.20

```
int[] marks={10,20};
try
{
    res = x / y;          // divide by 0
    Console.WriteLine("Result is : "+res);
    res = marks[2] / marks[0];    //index out of range
    Console.WriteLine("Result is : "+res);
}
catch (DivideByZeroException e)
{
    Console.WriteLine("Divide by 0");
}
catch (IndexOutOfRangeException e)
{
    Console.WriteLine("Array index out of range");
}
Console.WriteLine("End of the program.");
}
```

Whenever an exception occurs an exception object is created and the control jumps out of the try block and transfers to the appropriate catch block, whose exception object matches with the exception object created by the try block. Rest of the catch blocks will not be executed.

3.7.3 Use of finally Block

Appart from the try....catch blocks, C# also provides us with an optional finally block. The statements in this block, get executed regardless of the control flow. This means that the code in the finally block gets executed irrespective of whether an exception is raised or not. If any exception occurs inside the try block, the control transfers to the appropriate catch block and later to the finally block. But in C#, both catch and finally blocks are optional. The try block can exist:

- With one or more catch blocks
- With a finally block
- With both catch and finally blocks.

If there is no exception occurred inside the try block, the control directly transfers to finally block. We can say that the statements inside the finally block executes always.

Syntax: try...catch...finally Blocks

```
try
{
```



```

        //Program Logic
    }
    catch(ExceptionType1 ExceptionObject)
    {
        //Error Handling Code
    }
    catch(ExceptionType2 ExceptionObject)
    {
        //Error Handling Code
    }
    .
    .
    .
    catch(ExceptionTypeN ExceptionObject)
    {
        //Error Handling Code
    }
    finally
    {
        //clean up process or finally code
    }

```

Syntax: try...finally Blocks

```

try
{
    //Program Logic
}
finally
{
    //clean up process or finally code
}

```

Example: try...catch...finally Blocks

```

class HandlingExceptions
{
    public static void Main(String[] ar)
    {

```



```

int[] marks={10,20};
try
{
    int res = marks[2] / marks[0];    //index out of range
    Console.WriteLine("Result is : "+res);
}

catch (IndexOutOfRangeException e)
{
    Console.WriteLine("Array index out of range");
}
finally
{
    Console.WriteLine("Finally Block Executes Always!");
}
Console.WriteLine("End of the program.");
}

```

3.7.4 Catching all Exceptions

By providing a catch block without brackets or arguments, we can catch all exceptions occurred inside a try block. Even we can use a catch block with an **Exception** type parameter to catch all exceptions happened inside the try block since in C#, all exceptions are directly or indirectly inherited from the **Exception** class.

Syntax-1: try...catch to catch all exceptions

```

try
{
    //Program Logic
}
catch
{
    //Error Handling Code
}

```

Syntax-2: try...catch to catch all exceptions

```

try
{

```



```
//Program Logic
}
catch(Exception object)
{
    //Error Handling Code
}
```

Example-1: Catching all Exceptions

class HandlingExceptions

```
{
    public static void Main(String[] ar)
    {
        int[] marks={10,20};
        try
        {
            int res = marks[2] / marks[0]; //index out of range
            Console.WriteLine("Result is : "+res);
        }
        catch
        {
            //catch all types of exception
            Console.WriteLine("Array index out of range");
        }
    }
}
```

This can also be done as follows:

Example-2: Catching all Exceptions

class HandlingExceptions

```
{
    public static void Main(String[] ar)
    {
        int[] marks={10,20};
        try
        {
            int res = marks[2] / marks[0]; //index out of range
            Console.WriteLine("Result is : "+res);
        }
    }
}
```



```

    }
    catch (Exception e)           //catch all types of exceptions
    {
        Console.WriteLine("Array index out of range");
    }
}

```

3.7.5 Standard Types of Exceptions in C#

There are two types of exceptions in c#.

- Exceptions generated by an executing program.
- Exceptions generated by the common language runtime.

System.Exception is the base class for all exceptions in C#. Several exception classes inherit from this class including **ApplicationException** and **SystemException**. These two classes form the basis for most other runtime exceptions. Other exceptions that derive directly from **System.Exception** include **IOException**, **WebException** etc.

The CLR throws **SystemException**. The **ApplicationException** is thrown by a user program rather than the runtime. The **SystemException** includes number of derived exception types. In Table 3-B, some of the exception types and their cause of occurrence are given.

Table 3-B

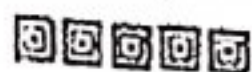
Exception Type	Cause of Occurrence
ArgumentException	It occurs when a method is called with an invalid argument.
ArgumentNullException	It occurs when a method argument is (unexpectedly) null
ArgumentOutOfRangeException	It occurs when an argument you passed to a method falls outside the allowed numeric range.
ArrayTypeMismatchException	This exception is thrown when an array element location is assigned to an object whose type is not compatible.
DirectoryNotFoundException	It is a type of file handling exception that is caused by a missing directory.
DivideByZeroException	It occurs when in a statement we attempted to divide an integer by 0.
FileNotFoundException	This exception is raised when we try to access a file from a particular location and that file does not exist at that location.
FormatException	This exception is thrown when the format of an argument does not meet the parameter specifications of the invoked method.

IndexOutOfRangeException	This exception occurs when we try to access an element at an index greater than the maximum allowable index of an array.
InvalidCastException	This exception is thrown for invalid casting or explicit conversion.
InvalidOperationException	This exception is thrown when a method call is invalid for the object's current state, regardless of any particular argument values. For example, reading an unopened file.
NotImplementedException	This exception is thrown when a requested method or operation is not implemented.
NullReferenceException	This exception is thrown when there is an attempt to dereference a null object reference.
OutOfMemoryException	The exception that is thrown when there is not enough memory to continue the execution of a program.
OverflowException	This exception thrown when an arithmetic casting or conversion operation in a checked context results in an overflow.
StackOverflowException	The stack has limited memory. This exception is thrown when the execution stack overflows by having too many pending method calls. It may occurs while recursion.
ObjectDisposedException	This exception is thrown when an operation is performed on a disposed object.

REVIEW QUESTIONS

1. What are known as three pillars of object-oriented programming?
2. What is encapsulation? What is its role in object-oriented programming?
3. What are the two major categories of class members?
4. Describe three different ways of providing values to instance variables of a class.
5. How are the static variables given initial values?
6. Why do we declare the data fields as public?
7. When do we use a private modifier to a data field?
8. What is the implication of declaring a member protected?
9. Describe two ways of accessing the values of data fields? Compare them.
10. What is a constructor? State three important characteristics of constructors? Why the usually declare constructors as public? Why do we need to use overloaded constructors?
11. When do we use static variables? When do we use static methods?

12. What is the difference between a static method and an instance method?
13. What does the keyword `this` refers to?
14. What is a static constructor? How is it different from a non-static constructor? What are the restrictions of static methods?
15. What are the restrictions of static methods?
16. What is a private constructor? When do we use a private constructor? What is a copy constructor? Why do we need such a constructor? What is finalization?
17. Give an example of typical use of `this` reference. What is class nesting? Give an example of typical use of class nesting.
18. Distinguish between constant members and read-only members.



4.8.2.3 ListBox Control

ListBox control is a collection of items from which the users can select one or more items. Items can be added to a ListBox control by using the Items property of it. The basic syntax to add a ListBox control to a web form using source code is as follows:

```
<asp:ListBox ID="ListBox1" runat="server" SelectionMode="Multiple">
<asp:ListItem>ListItem1</asp:ListItem>
<asp:ListItem> ListItem2</asp:ListItem>
</asp:ListBox>
```

We can also add items to the ListBox control dynamically through code. For example, in C#, this can be done as follows:

```
ListBox1.Items.Add("ListItem");
```

Table 4-L lists some commonly used properties of the ListBox control.

Table 4-L : Properties of the ListBox control	
Property	Description
Items	Used to add collection of items to the ListBox control.
Rows	Used to specify the number of items to be displayed in the list box. If actual list contains more rows (items) than the number of rows to be displayed, then a scroll bar is added.
SelectedIndex	Used to get or set the index of the currently selected item. If more than one item is selected, then the index of the first selected item is represented by this property. If no item is selected, the value of this property is -1.
SelectedValue	Used to get or set the value of the currently selected item. If more than one item is selected, then the value of the first selected item is represented by this property. If no item is selected, the value of this property is an empty string ("").
SelectionMode	It indicates whether a list box allows single selections or multiple selections to the user.

SelectedIndexChanged is the most commonly used event of the ListBox control. It raised on the server when the user selects a different item from list box. The basic C# syntax for SelectedIndexChanged event of a ListBox control is as follows:

```
protected void ListBox1_SelectedIndexChanged(object sender, EventArgs e) //Event Handler
{
    //Write Code Here
}
```

Program 4-17

Description of the Program 4-17: This program demonstrates how to add items in a ListBox control on Button click. Follow the steps given below.